

NAME : Y V BHAVANA

SRN: PES1UG24AM328

SEC: F

SE LAB 4 CHAT HISTORY

Lab_4_VibeCoding_Student_handout.pdfPDF

https://github.com/SETAPESU26/28_defender check this and guide wt should i do for SE lab guide me step by step

Bro, I checked your **exact assigned repo 28_defender** and the lab handout. You have a very manageable lab — **1 bug + 3 features**. The README specifically says to use an LLM iteratively and keep separate work for each task.

Your handout says the overall flow is: clone/fork → run the Python game → record 10 sec before → use prompts to fix/add features → record 10 sec after → push your repo with separate commits.

First understand what you have to do

Your repo contains:

- game.py
- README.md

The game is a **Defender-style Pygame game**. The README gives exactly **4 tasks**:

Task	What you need
1	Fix radar bug
2	Implement sky_color(wave)
3	Implement on_humanoid_rescued(humanoid)
4	Implement bonus_life_threshold()

And **DO NOT** create a PR to SETAPESU26. Your changes need to go to your own repo.

STEP 1 — Get the repo onto your laptop

Open **PowerShell**.

Go to wherever you want the project:

cd Desktop

Then clone:

```
git clone https://github.com/SETAPESU26/28_defender.git
```

Go inside:

```
cd 28_defender
```

Check:

```
dir
```

You should see:

```
game.py
```

```
README.md
```

```
.gitignore
```

STEP 2 — IMPORTANT: Make your own GitHub repo

Because your lab says **push to your personal account** and don't PR the main SETAPESU26 repo.

Go to GitHub and create a **new repository**.

For example:

```
28_defender
```

or

```
SE-Lab-4-Defender
```

Make it **public** if your faculty needs to check it.

Don't add README/gitignore when creating it because you already have them locally.

Then we'll connect your local project to your repo.

STEP 3 — Run the ORIGINAL game FIRST

This is important because your lab specifically asks for a **10-second video before making changes**.

Check Python:

```
python --version
```

The repo expects Python 3.10+.

Install pygame:

```
pip install pygame
```

Then:

```
python game.py
```

The game should open.

Controls:

- **Left / Right** → thrust
- **Up / Down** → altitude
- **Space** → fire
- **R** → reset

These are specified in your repo README.

STEP 4 — Record the BEFORE video

Do NOT modify the code yet.

Record around **10 seconds** of the original game.

Try to show the radar at the top.

The README says the radar currently has the deliberate bug: objects are clustered around the player's position instead of being shown according to their actual world position.

So your before video should ideally show:

Game running → move player → radar showing the incorrect/clumped positions.

Save it as something like:

before.mp4

STEP 5 — Open game.py

Use VS Code:

code .

Now **don't start randomly changing code.**

We're going to do the assignment exactly task-by-task.

TASK 1 — Fix the radar

The README explains the bug very clearly:

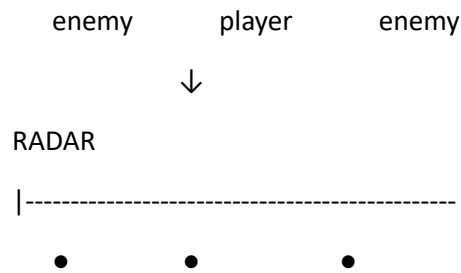
Radar should represent the entire wrapping world.

Currently it incorrectly uses the player's relative/screen position.

The required behavior is:

WORLD

|-----|



The radar should show **absolute world positions**, not positions relative to the player's viewport.

This is where your Vibe Coding comes in

Don't just ask:

"Fix this code."

Give the AI a good prompt.

Use this as your **Task 1 prompt**:

I am working on the Defender Pygame lab.

Please inspect game.py and specifically analyze the draw_radar function.

The radar is currently incorrect because objects appear clustered around the player's position. According to the README, the radar must represent the entire wrapping world using each object's absolute world-space x coordinate, not screen-space/player-relative coordinates.

Fix **ONLY** the radar bug.

Do not modify unrelated game mechanics or features.

Explain what is wrong, show me the relevant change, and make the smallest safe code modification.

After ChatGPT gives you the change

Read it.

This is important because your lab says you should use the LLM as a debugging/pair-programming partner and critically review its suggestions.

Then run:

python game.py

Test the radar.

STEP 6 — Commit Task 1

Once Task 1 works:

```
git status
```

Then:

```
git add game.py
```

Commit:

```
git commit -m "Fix radar world position bug"
```

This gives you a **separate commit for Task 1**, which your handout requires.

TASK 2 — `sky_color(wave)`

Now we implement the second empty feature.

The README says:

```
sky_color(wave)
```

is called every frame and should return:

```
(r, g, b)
```

or:

```
None
```

The suggested idea is to change/brighten the sky as the waves increase.

Use this prompt:

Now work on Task 2 only.

Inspect the existing `sky_color(wave)` function in `game.py`.

Implement it so the background color changes progressively as the wave number increases.

Keep the implementation simple and compatible with Pygame RGB colors.

Do not modify the radar, player controls, enemy AI, scoring, or any other functionality.

Explain the implementation briefly and make the code change.

Then run:

```
python game.py
```

You should be able to see the background changing as waves progress.

Commit:

```
git add game.py
```

```
git commit -m "Add progressive sky colors"
```

TASK 3 — Humanoid rescue feature

The third function is:

```
on_humanoid_rescued(humanoid)
```

The README tells you exactly what happens before this function is called:

- player catches a falling humanoid
- 500-point bonus is added
- humanoid is placed safely on ground
- this function is called

The suggested feature is something like:

+500 popup

or

short invulnerability reward.

I'd choose **the +500 popup** because it's easier, safer and easier to demonstrate in your final video.

Use this prompt:

Now implement Task 3 only.

Inspect `on_humanoid_rescued(humanoid)` in `game.py`.

When the player rescues a falling humanoid, the game already awards the 500-point bonus and places the humanoid safely on the ground.

Add a simple visible "+500" rescue feedback effect, such as a temporary popup near the rescued humanoid.

Use the existing game architecture and Pygame rendering style.

Do not change the existing scoring logic or other game mechanics.

Keep the change minimal and explain what you changed.

Then test:

```
python game.py
```

Try to rescue a humanoid.

If it works, commit:

```
git add game.py
```

```
git commit -m "Add humanoid rescue feedback"
```

TASK 4 — Bonus life

Last one:

```
bonus_life_threshold()
```

The README actually suggests:

```
return 10000
```

That means a bonus life is awarded when the score crosses multiples of **10,000**, with the existing bookkeeping already handling whether that threshold has been awarded.

Use the LLM prompt anyway because this is a Vibe Coding assignment:

Now implement Task 4 only.

Inspect `bonus_life_threshold()` in `game.py`.

The existing update logic already handles awarding a bonus life when the score crosses a multiple of the returned threshold.

Implement this function so that players receive a bonus life for every 10,000 points.

Return the appropriate integer threshold.

Do not modify the existing bonus-life bookkeeping or unrelated game logic.

Explain the change briefly.

Then:

```
python game.py
```

Commit:

```
git add game.py
```

```
git commit -m "Add 10000 point bonus life threshold"
```

STEP 7 — Check your commits

Run:

```
git log --oneline
```

Ideally you'll have something like:

```
xxxxxxx Add 10000 point bonus life threshold
```

```
xxxxxxx Add humanoid rescue feedback
```

```
xxxxxxx Add progressive sky colors
```

```
xxxxxxx Fix radar world position bug
```

```
xxxxxxx Initial commit
```

This is exactly why we're doing separate commits.

Your handout explicitly says:

"Have Separate Commits for each task."

STEP 8 — Connect to YOUR GitHub repo

Check current remote:

```
git remote -v
```

It will probably show the SETAPESU26 repository.

Remove it:

```
git remote remove origin
```

Then add your own repository:

```
git remote add origin YOUR_GITHUB_REPO_URL
```

For example:

```
git remote add origin https://github.com/YOUR_USERNAME/28_defender.git
```

Check:

```
git remote -v
```

Make sure it shows **your account**, not SETAPESU26.

Then:

```
git branch -M main
```

```
git push -u origin main
```

STEP 9 — Make your Lab-4 submission folder

Your handout says the Lab-4 submission needs the before/after videos, updated code and chat history.

I'd make your personal lab repo look like:

SE-Lab/

|

├── Lab-4/

| ├── before.mp4

| ├── after.mp4

| └── chat-history.pdf

|

├── game.py

├── README.md

└── .gitignore

However, **your repository README's submission checklist says the required submission is specifically the 10-second before video, 10-second after video, and the Chat/LLM page link with complete chat history.**

So follow whatever your faculty's submission portal specifically asks for if it differs.

STEP 10 — FINAL 10-second video

Now run:

```
python game.py
```

Your final video should demonstrate the changes.

Try to show:

1. Radar

Move around and show that objects are distributed across the radar according to their world positions.

2. Sky

Show the changing background between waves if possible.

3. Rescue

Rescue a humanoid and show the:

+500

feedback.

4. Bonus life

This one is difficult to demonstrate naturally because you need 10,000 points.

Don't fake it. Your code implementation is what matters; the video can focus on the more visually demonstrable changes.

Save:

after.mp4

One VERY important thing

Your lab is called **VibeCoding**, and the handout specifically says:

"Write prompts and fix the broken code"

"Write prompts to add all the features listed in the ReadMe file one by one."

So **keep this chat history**.

Don't just copy code from somewhere and submit it.

Your sequence should be:

Prompt → AI suggestion → You review → Modify → Test → Commit

for each task.

That gives you a clean story if your faculty asks:

"How did you use Vibe Coding?"

Your whole lab in one checklist

- ☐ Clone 28_defender
- ☐ Create your own GitHub repo
- ☐ Install pygame
- ☐ Run original game
- ☐ Record 10-sec BEFORE video

☐ Task 1 — Fix radar

☐ Test

☐ Commit

☐ Task 2 — sky_color()

☐ Test

☐ Commit

☐ Task 3 — on_humanoid_rescued()

☐ Test

☐ Commit

☐ Task 4 — bonus_life_threshold()

☐ Test

☐ Commit

☐ Run final game

☐ Record 10-sec AFTER video

☐ Push to YOUR GitHub

☐ Export/save complete ChatGPT history

☐ Put required Lab-4 material in your submission

The repo itself confirms that these are the four required code tasks and that the project is a single-file Pygame Defender implementation.

And don't worry — we can do this together one task at a time. Start with **Step 1–4**, run the original game.py, and then send me a screenshot of your VS Code/PowerShell if you get stuck. I can guide you through **Task 1 line-by-line** without you messing up the repo.